

Text Property Graphs for Security Text

1 Introduction

Security text is not only a sequence of words. A CVE description, an exploit report, or a vendor advisory states which vulnerability is being discussed, which software is affected, what version is vulnerable, how the attack works, what the impact is, and what mitigation is recommended. These facts are not independent keywords. They are connected by syntax, sentence order, entity references, discourse markers, and security relations.

The Text Property Graph (TPG) was designed to preserve that structure. It represents text as a directed, typed, attributed property graph. Nodes represent linguistic or domain-specific objects: documents, sentences, tokens, entities, predicates, CVE identifiers, software names, attack vectors, impacts and remediations. Edges represent typed relations between those nodes: containment, dependency, token order, sentence order, coreference, predicate-argument binding, discourse relation, entity relation, and optional security-specific relations.

The design is inspired by the Code Property Graph (CPG). A CPG combines several views of source code (syntax, control flow, data flow) into one graph. The TPG applies the same idea to natural-language text. Instead of keeping dependency parsing, named entity recognition, coreference, discourse structure and domain relations as separate outputs, the TPG stores them as different edge types over one shared graph.

This chapter explains the problem that motivates TPG, the core mechanics of the representation, the design choices we made along the way (GNN over standard models, TPG over plain transformers, multiview fusion over a single GNN), and the linguistic backend (spaCy) that turns raw text into the initial nodes and edges. The claims here are limited to what is implemented and documented in the project.

2 Why We Chose a GNN-Based Solution for EPSS

EPSS prediction is a binary or soft-label classification problem on a single CVE: given everything we know about the CVE, predict the probability that it will be exploited in the wild. The natural question is: what kind of model fits this best?

A CVE record is not one thing. It is a small bundle of heterogeneous inputs. There is a free-text description (one or two sentences). There is a CVSS vector (eight categorical metrics). There is a CWE classification (a list of weakness IDs). There are publication dates, exploit-code flags, and source-mention counts. And, in our datasets, there are LLM summary columns. Some of these inputs are tabular. Some are sequential text. Some are graph-shaped relations between entities mentioned in the text.

We chose a graph neural network because it is the model class that handles heterogeneous, structurally varied input most directly:

- A GNN’s input is a graph. We can represent each CVE as a graph whose nodes carry features and whose edges carry typed relations. The shape and size of the graph can vary per CVE; the GNN does not require a fixed input length.
- A GNN learns from local neighbourhoods through message passing. This means the prediction for a CVE depends not only on which words appear, but on which entities are connected to which other entities. That is closer to how a security analyst reads an advisory.
- A GNN extends naturally to a hybrid model. We can attach contextual text embeddings to nodes and concatenate the pooled graph embedding with a tabular feature vector inside the same model. The graph branch sees the typed relational structure, the tabular branch sees the structured CVE metadata, and the head sees both.

A second reason is interpretability. With a GNN over a TPG, we can trace which nodes received the highest attention weight and which edges carried the most useful signal. With a flat sequence model the same question becomes much harder.

A third reason is room to grow. If we later want to link a CVE to its patch commit, to a CWE knowledge graph, or to source-code constructs extracted by Joern, we can add new node types and edge types without changing the rest of the pipeline.

3 Why We Chose TPG to Drive the GNN

Once we settled on a GNN, the next question was: what graph?

A few options were on the table. We could have used a dependency parse alone. We could have used an Abstract Meaning Representation (AMR) graph. We could have built a knowledge graph of typed entity triples. Each of these is a single-view representation: a parse captures only syntax, AMR captures only predicate-argument structure, a knowledge graph captures only typed relations between named entities.

The Code Property Graph (CPG) introduced by Joern for source-code analysis offered a different idea. A CPG keeps several views of a program (Abstract Syntax Tree for syntax, Control Flow Graph for execution order, Reaching-Definitions Graph for data flow, Control Dependence Graph for guarded statements) as different edge types over one shared node set. The benefit is that a query can cross views without joining separate graphs.

We borrowed that pattern for text. The TPG keeps the four CPG-style views as different edge groups over the same nodes:

CPG view	Text-domain analogue used in the TPG
Abstract Syntax Tree (AST)	Dependency parse, edge type <code>DEP</code>
Control Flow Graph (CFG)	Token, sentence, paragraph order: <code>NEXT_TOKEN</code> , <code>NEXT_SENT</code> , <code>NEXT_PARA</code>
Reaching-Definitions Graph	Coreference, edge type <code>COREF</code>
Control Dependence Graph	Discourse and rhetorical relations: <code>RST_RELATION</code> , <code>DISCOURSE</code>

Table 1: The four CPG views and their text-domain analogues in the TPG.

To these we added structural edges (`CONTAINS`, `BELONGS_TO`) and predicate-argument edges (`SRL_ARG`, `ENTITY_REL`, `SIMILARITY`, `AMR_EDGE`) that are not in the CPG but are useful for text. The full mapping is recorded in the schema as a machine-readable dictionary so the correspondence is unambiguous.

The reason this design helps is the same reason it helped Joern. A single query (or, in our case, a single GNN forward pass) can use information from all views at once. The model is not forced to pick between syntax, sequence, coreference and discourse. It sees them together as different edges in the same graph.

4 The TPG Is Generic, Not Just a Security Representation

We use the TPG for security text in this project, but the schema itself is not security-specific. The bottom layer is generic English NLP. The 13 base node types (DOCUMENT, PARAGRAPH, SENTENCE, TOKEN, ENTITY, PREDICATE, ARGUMENT, CONCEPT, NOUN_PHRASE, VERB_PHRASE, CLAUSE, MENTION, TOPIC) and the 13 base edge types (DEP, NEXT_TOKEN, NEXT_SENT, NEXT_PARA, COREF, SRL_ARG, AMR_EDGE, RST_RELATION, DISCOURSE, CONTAINS, BELONGS_TO, ENTITY_REL, SIMILARITY) describe any English text. They make no assumption about the topic.

The security part is a Level 2 overlay built on top of that generic layer. It adds 12 domain node types (CVE_ID, CWE_ID, SOFTWARE, VERSION, COMPONENT, CODE_ELEMENT, ATTACK_VECTOR, IMPACT, REMEDIATION, THREAT_ACTOR, VULN_TYPE, SEVERITY) and 10 SEC_* edge types. The overlay is optional. When it is not loaded, the schema is exactly the generic Level 1 schema and works on any English text without changes.

A different domain can be added the same way:

- For **clinical text**, an overlay would add node types such as DRUG, DOSE, ROUTE, CONDITION, SYMPTOM and edge types such as TREATS, HAS_DOSE, ADMINISTERED_BY, CAUSES_SIDE_EFFECT.
- For **legal text**, an overlay would add STATUTE, CASE, PARTY, HOLDING with edges CITES, OVERRULES, DISTINGUISHES.
- For **scientific text**, an overlay would add METHOD, DATASET, RESULT, CITATION with edges EVALUATES_ON, COMPARES_AGAINST, REPORTS_RESULT.

In every case the generic layer (sentences, tokens, dependency parse, coreference, discourse) is shared. Only the typed entities and the typed relations between them change. Anything built on top of the generic layer (the GNN backbone, the multiview decomposition, the hybrid head) keeps working without modification, because the model consumes a typed property graph regardless of which types are present.

In other words, the TPG is a general-purpose representation for information processing on text. The security extension is one instantiation of it. The same machinery would carry medical case notes, legal opinions, scientific abstracts, or any other domain whose meaning depends on typed entities and typed relations between them.

5 Why a GNN over a TPG, Not a Pure Transformer or a Pure Tabular MLP

A reasonable alternative would have been to skip the graph and use a standard NLP transformer (BERT or SecBERT) on the description plus a tabular MLP on the CVSS one-hot and the count features, then concatenate the two embeddings. Why did we go further?

Pure transformer on text alone. A transformer reads the description and produces a fixed-length embedding. That embedding is dense and contextual, but it does not expose any structure. If we want to know which token in the description corresponds to the CVE identifier, the affected software, or the attack vector, the transformer does not say. The information is implicit in the attention weights but is not surfaced as queryable structure. Adding the CVSS features as tabular input next to a transformer embedding works (and is in fact what our model does in the tabular branch), but it gives up the relational structure inside the text. A typical CVE description is short (one or two sentences) and dense with relations, so losing that structure has a real cost.

Pure tabular MLP with one-hot encodings. We could also have skipped the text altogether and trained an MLP on just the CVSS one-hot, the CWE multi-hot, and the count features. This is the simplest and most leaky-resistant baseline. The risk is the opposite of the transformer case: we drop everything the description says about the vulnerability. Two CVEs with the same CVSS vector and no CWE list look identical to a tabular MLP. The description is what distinguishes them.

What we actually do. The TPG-plus-GNN-plus-tabular hybrid sits between these extremes. The graph branch reads the description through the TPG, gets a 768-dim SecBERT embedding per token (so the contextual signal is preserved on the nodes), and runs message passing over typed edges. The tabular branch reads the CVSS one-hot, the CWE multi-hot, age, and counts as a 55-dim vector. The two branches meet in the prediction head. Neither the structure nor the metadata is thrown away.

The 20-run text-source ablation reports that on the in-distribution corpus the description carries the predictive signal and the LLM summaries do not add to it. That is an empirical finding about the data, not an indictment of the architecture. The architecture supports a clean reading of the result because the model produces interpretable per-variant deltas.

6 Multiview GNN, and Why “Multiview”

The GNN backbone we use is called the Multiview GNN.

What “multiview” means here. A view is a subset of the edge-type vocabulary. The model partitions the 13 generic edge types (and the 10 `SEC_*` edge types when security edges are enabled) into five groups, called views:

View	Edge types in the view
Syntactic	DEP, CONTAINS, BELONGS_TO
Sequential	NEXT_TOKEN, NEXT_SENT, NEXT_PARA
Semantic	COREF, SRL_ARG, AMR_EDGE
Discourse	RST_RELATION, DISCOURSE, ENTITY_REL, SIMILARITY
Security	all <code>SEC_*</code> types (only present when security edges are enabled)

Table 2: The five edge-type views used by the multiview GNN.

For each view, the model runs a separate Gated Graph Convolution (GGNN) restricted to that view’s edges. So the syntactic view sees only `DEP`, `CONTAINS` and `BELONGS_TO`; the sequential view sees only the three `NEXT_*` edges; and so on. This gives one node embedding per view. A node-conditioned attention fusion then combines the five view-specific node embeddings into a single

embedding per node. After that, mean-and-max pooling over nodes produces the graph embedding that the prediction head consumes.

Why this is called multiview. The term comes from multi-view learning, where the same input is encoded through multiple distinct “views” (different feature spaces, different modalities, or different relation types) and the resulting representations are combined. In our case the views differ in which edge types are visible. The same node set is shared, but each view’s GNN sees a different adjacency.

What inspired it. Two things. First, Joern’s CPG explicitly separates AST, CFG and DDG edges and uses them differently in queries. We wanted the same separation at the model level: a single shared GNN that mixes all edge types together would force syntax, sequence and coreference through the same parameters, even though they describe quite different relationships. Second, in the SemVul project (the predecessor of this work) the multiview decomposition gave a measurable improvement on the CPG side. We carried the same architecture across to the TPG, with the views replaced by their text-domain analogues.

Why it is the right shape for the TPG. Adding security edges expands the edge vocabulary from 13 to 23 types. With a single-view GNN, those 10 new types would have to share message passing parameters with everything else. With the multiview design, the security view is a separate encoder; if it has zero edges (because `-include-security-edges` is off) the encoder is the identity and contributes nothing, and if it has edges it learns its own parameters without disturbing the other four views. That makes the overlay clean and reversible.

7 spaCy: How the Linguistic Backend Works and What We Take From It

The TPG generic layer is built by a spaCy frontend. spaCy is what turns raw text into the initial DOCUMENT, PARAGRAPH, SENTENCE, TOKEN, ENTITY, PREDICATE, ARGUMENT, NOUN_PHRASE, VERB_PHRASE and CLAUSE nodes, and the linguistic edges between them.

7.1 What spaCy Is and How It Processes Text

spaCy describes itself as “Industrial-strength Natural Language Processing in Python”.¹ It ships pre-trained pipelines for English (and many other languages) that perform tokenisation, part-of-speech tagging, dependency parsing, lemmatisation, named entity recognition, sentence segmentation, and noun-phrase chunking, all in one pass.

The processing flow is:

1. Raw text comes in as a string.
2. The tokenizer splits it into tokens. spaCy’s tokenizer is non-destructive: the tokenised Doc can reconstruct the original input character for character. This matters for the TPG because we anchor every node to character offsets, and we need those offsets to line up with the SecBERT embedder.

¹<https://spacy.io/>

3. Statistical components annotate each token: POS tag, lemma, dependency-parse head, dependency relation label, NER tag, IOB tag.
4. Sentence boundaries are emitted as `Doc.sents`.
5. Noun-phrase chunks are emitted as `Doc.noun_chunks` when a parser is present.

The frontend loads the English pipeline `en_core_web_sm` when available. If that model is not installed it falls back to a blank English pipeline plus a sentencizer. The fallback can split sentences but does not provide the parser, the lemmatiser, or the NER. A flag is checked before every step that needs those components, so the TPG degrades gracefully.

7.2 What Counts as “NLP” Here

When we say spaCy gives us NLP, we mean it provides four concrete linguistic annotations on top of the raw token sequence:

- **Tokenisation and POS tagging.** Each token gets a surface form, a lemma, and a part-of-speech tag.
- **Dependency parsing.** Each token is linked to a syntactic head, with a labelled relation (`nsubj`, `dobj`, `amod`, `prep`, etc.). This is what gives the TPG its DEP edges.
- **Named entity recognition.** Spans of tokens are classified as PERSON, ORG, GPE, PRODUCT, DATE, etc. These spans become ENTITY nodes in the TPG.
- **Sentence and noun-phrase segmentation.** Sentences become SENTENCE nodes; noun chunks become NOUN_PHRASE nodes, attached to their constituent tokens via BELONGS_TO edges.

These are statistical annotations produced by the trained pipeline. The TPG then organises them into typed graph structure. spaCy contributes the per-token signal; the TPG contributes the graph shape.

7.3 The Exact Features We Take From spaCy

The frontend extracts the following features and turns them into nodes, properties or edges.

After this, the security frontend overlays domain entities (CVE IDs, CWE IDs, software, versions, attack vectors, impacts, severity, remediations) on top of the spaCy graph. Then the SecBERT embedder runs over the raw text, aligns its WordPiece sub-tokens to the spaCy tokens via character offsets, and attaches a 768-dim contextual vector to every TOKEN node (with mean-pooled versions on SENTENCE and ENTITY).

7.4 Why spaCy Specifically

spaCy gives us the four things we need in one consistent Doc object: tokenisation, POS and dependency parsing, NER, and sentence segmentation. Other libraries provide some of these (NLTK has tokenisation and POS; Stanza has dependency parsing); spaCy provides all of them with reasonable accuracy and a stable API, and the character offsets line up exactly with the BERT embedder we use later. The TPG schema is decoupled from spaCy (the frontend class is one of several possible frontends), so a different parser could be substituted, but the mapping from spaCy outputs to TPG structure is the one that the project actually implements and runs.

8 How Security Entities and SEC_* Edges Are Extracted

Once spaCy has built the linguistic skeleton, the security frontend runs a second pass over the same text and adds domain entities. The extraction is deliberately rule-based: regular expressions for patterns with stable surface forms, keyword dictionaries with longest-first matching for everything else. Rules are precise on canonical advisories and easy to audit. For surface forms that the rules miss, a SecBERT-similarity step in the hybrid frontend adds candidate entities, with rule entities winning on overlap.

8.1 The Regular Expressions

The patterns below are shown literally (in monospace, without LaTeX interpretation). They are case-insensitive unless noted.

CVE identifier.

```
CVE[<six Unicode hyphens>\-]\d{4}[<six Unicode hyphens>\-]\d{4,}
```

Matches the canonical ASCII form (CVE followed by a hyphen, year, hyphen, four-or-more digits) and the same form written with non-ASCII hyphens. The character class contains all six Unicode hyphen code points in the range U+2010 to U+2015 along with the ASCII hyphen, because LLM-generated summaries often emit a non-ASCII hyphen between the year and the sequence number.

CWE identifier.

```
CWE-\d{1,4}
```

Plain ASCII hyphen, weakness identifier of one to four digits.

Software version (semantic-version shape).

```
\b\d+\.\d+(?:\.\d+)*(?:-[a-zA-Z0-9]+)?\b
```

Catches 1.2.3, 1.0.0-rc1, 4.18.0-372, etc. Anchored on word boundaries. A separate context filter rejects numerals that look like CVSS scores when the surrounding sentence mentions “cvss” or “score”.

CVSS score in context.

```
\b(?:CVSS[:\s]*v?\d?\.\d?[:\s]*[/-]?[s*])?(\d+\.\d+)(?:[s*/s*10])?\b
```

A composite pattern that matches CVSS 9.8, CVSS:9.8, CVSS v3 9.8, 9.8/10, and parenthetical scores near the word “CVSS”.

Severity with context.

```
\b(critical|high|medium|moderate|low|informational)\s+
(severity|risk|impact|cvss\s*score|cvss|score|
vulnerability|flaw|priority|confidence)\b
```

Requires the severity adjective to be followed by a security-context noun. The required second token is what stops phrases like “high availability” or “low memory” from being tagged as severities.

Bare severity (fallback).

```
\b(critical|high|medium|moderate|low)\b(=?\s*([.,;:!\?\)\]]|$))
```

A looser fallback that fires only when the severity adjective is immediately followed by punctuation or end-of-clause. The extractor runs this fallback only inside clauses that already contain a security keyword within ± 60 characters.

Code element.

```
\b(strcpy|strcat|sprintf|gets|scanf|memcpy|memmove|  
  malloc|free|printf|fprintf|eval|exec|system|popen|  
  fork|[a-z_][a-z0-9_]*\(\))\b
```

A list of dangerous C-stdlib functions and scripting eval/exec/system names, plus the generic name() shape that catches any other identifier written with parentheses.

Remediation actions (about fifteen patterns). The remediation extractor compiles a list of small patterns and tags each match with a remediation type. The most common ones are:

upgrad\w*\s+to\s+(?:version\s+)?(\S+)	→ upgrade
update\s+to\s+(?:version\s+)?(\S+)	→ upgrade
patch\w*\s+(?:by\s+)?(?:applying installing)	→ patch
apply\s+(?:the\s+)?patch	→ patch
security\s+patch	→ patch
patched\s+version	→ patch
install(ing)?\s+the\s+update	→ update
\bfix(?:ed)?\s+in\s+(?:version\s+)?(\S+)	→ update
workaround	→ workaround
mitigat\w+	→ mitigation
remediat\w+	→ mitigation
disabl\w+\s+\w+	→ disable_feature
restrict\w*\s+\w+	→ restrict_access
firewall\s+(?:rule block)	→ network_block

Vendor and product fallback. A pattern that catches title-case “Vendor Product” mentions near a version number. Used to recover niche products that are not in the curated software dictionary (for example, proprietary appliances and in-house tooling).

8.2 The Dictionary-Based Extractors

Some entities cannot be captured cleanly with a regex because their surface forms are open-ended. For these we use curated keyword dictionaries with longest-first matching:

- **Software products.** A set of about 140 known names (Apache, nginx, OpenSSL, Linux kernel, WordPress, Spring Boot, Telerik, Confluence, vCenter, FortiOS, Palo Alto PAN-OS, etc.). Multi-word names like `apache http server` are matched before `apache` so the longer span wins. Word boundaries are anchored on either side so `git` does not match `github`.

- **Code constructs.** A focused list of around 35 security-relevant phrases (`deserialization`, `template engine`, `xml parser`, `session token`, `prepared statement`, `authorisation check`, etc.) that appear in CVE descriptions but are rarely written with parentheses.
- **Attack vectors.** About ten keyword phrases (`remote`, `local`, `authenticated user`, `user-supplied input`, etc.).
- **Impact types.** About 25 phrases that name what an attacker gains (`remote code execution`, `denial of service`, `information disclosure`, `privilege escalation`, etc.).
- **Vulnerability types.** About 25 classification phrases (`buffer overflow`, `cross-site scripting`, `sql injection`, `path traversal`, `use after free`, `race condition`, etc.). The vulnerability-type extractor runs first and shares its claimed spans with the impact and attack-vector extractors, so a phrase that appears in two dictionaries is tagged once with the most specific label.

For each dictionary the extractor sorts entries longest-first, compiles a word-boundary-anchored regex per entry, and keeps a `seen_spans` list so two patterns cannot claim the same character range twice. Every entity that survives is added as an ENTITY node in the graph, tagged with its `domain_type` property, and linked to its containing SENTENCE via a CONTAINS edge.

8.3 From Entities to SEC_* Edges

The entities are the nodes. The relations between them are edges. The security-relations pass walks the entity nodes and emits ten typed SEC_* edges:

The pass is deliberately conservative. It mostly looks for source and target entities inside the same sentence (using each entity’s `sentence-index` property), and only links them if both are present. This keeps the pass cheap and avoids spurious cross-document edges. For relations where the typical writing pattern crosses one sentence boundary (mitigation phrases at the end of an advisory, for example) the pass widens the window slightly. The result is a small, high-precision set of typed edges that the multiview GNN’s security view can consume directly.

8.4 Why Rule-Based Extraction Is Used Here

Two reasons. First, the surface forms that matter (CVE-IDs, CWE-IDs, semantic versions, CVSS scores) are syntactically rigid. Regular expressions are the natural fit and produce near-perfect precision on canonical advisories. Second, the extracted entities feed an interpretable downstream pipeline: a graph-level decision can be traced back to specific entity nodes, and an analyst can audit the extraction by looking at the regex that produced each node. A neural extractor would make the same decisions opaque.

The trade-off is recall on novel surface forms. The hybrid frontend addresses this by adding a SecBERT-similarity pass that recovers entities the rules miss, with rule entities winning on overlap so the audit trail stays intact.

9 Training Hyperparameters and Configuration

The training command used for the 20-run text-source ablation and the NVD/KEV runs is:

```
python -m epss.run_pipeline -backbone multiview -hybrid -label-mode soft -epochs
    100 -no-epss-feature ...
```

The ... is the variant-specific flag (`-summary-source`, `-summary-only-tpg`, `-include-summary-in-tpg`, etc.). The defaults for all other hyperparameters are:

The training loop has the following pieces:

- **Loss.** Binary cross-entropy with logits. For the soft EPSS target, BCE works on continuous labels in $[0, 1]$ as well as on hard $\{0, 1\}$ labels. A positive class weight is set to the negative-to-positive ratio in the training set so that the rare high-EPSS class gets enough gradient signal.
- **Optimiser.** AdamW with the learning rate and weight decay above.
- **Scheduler.** Reduce-on-plateau, mode max, factor 0.5, patience 5, minimum learning rate 10^{-6} , monitoring validation PR-AUC.
- **Gradient clipping.** Gradient-norm clipping at 1.0 after each backward pass.
- **Train loader.** Shuffle on, drop-last on. Dropping the last partial mini-batch avoids a known crash when the final batch has only one example and the model contains a BatchNorm layer in the prediction head (BN needs more than one element per channel).
- **Class weights.** Computed once from the full dataset before the splits are created. This is approximately the train-set ratio because the splits are stratified.
- **Splits.** If no external test file is provided, the dataset is split into train / val / test by class-stratified random sampling. If an external test file is provided, the internal split is train (85%) / val (15%) and the external file becomes the test set.

For the leakage-free configuration used in the 20-run ablation, the relevant input dimensions are:

The companion document on the multiview formula writes out the encoder, the node-conditioned attention fusion, and the hybrid prediction head with all dimensions.

10 Limitations

Here are a few limitations:

- Coreference is heuristic in the current enrichment pass. It uses recency and basic agreement, not a full neural coreference resolver.
- Discourse relations are marker-based. They catch explicit markers (“however”, “because”, “subsequently”) but do not perform full discourse parsing.
- Security entity extraction depends on regular expressions, keyword dictionaries, and SecBERT-similarity thresholds. New surface forms (Telegram threat-intelligence chatter, for example) are recovered less reliably than NVD-style advisories.
- Security edges are opt-in. If the pipeline does not enable them and the model’s edge-type vocabulary is not expanded to 23, the GNN cannot use `SEC_*` edges as first-class types.
- TPG is a representation. It improves the structural information available to the model. It does not guarantee a better PR-AUC on every downstream task. The 20-run ablation showed this plainly: the LLM summaries did not beat description-only on this in-distribution corpus.

11 Conclusion

The Text Property Graph is a structured, typed, attributed graph representation of text. The central idea is borrowed from the Code Property Graph: keep several views of the input in one graph rather than as separate analyses. For text those views are dependency structure, sequence, coreference, predicate-argument structure, discourse, entity relation, and optional security relations.

We chose a GNN over the TPG because the input is heterogeneous and relational. We chose the TPG over a single-view alternative because the CPG inspiration generalises naturally to text. We chose the multiview backbone because different edge groups deserve different encoders. We chose spaCy as the linguistic backend because it gives us tokenisation, parsing, NER and sentence segmentation in one consistent Doc object whose character offsets line up with the SecBERT embedder.

For security text, this combination preserves the relations that matter (which CVE affects which software, with which attack vector, causing which impact, mitigated by which fix) and feeds them to the model as typed graph structure rather than as flat embeddings. The empirical results show what this gives us in practice: a strong description-driven baseline whose signal does not get displaced by adding more text, and a representation that is ready for the cross-modal and temporal experiments planned next.

spaCy annotation	Used for	TPG output
<code>Doc.sents</code> (sentence boundaries)	Splitting paragraphs into sentences	SENTENCE nodes; CONTAINS edges from paragraph to sentence
Token text and <code>token.idx</code>	Per-token surface and character offset	TOKEN nodes; <code>char_start</code> / <code>char_end</code> properties
<code>token.lemma_</code>	Canonical form of each token	<code>lemma</code> property on TOKEN
<code>token.pos_</code>	Part-of-speech tag	<code>pos_tag</code> property on TOKEN
<code>token.dep_</code>	Dependency relation label (<code>nsubj</code> , <code>dobj</code> , etc.)	<code>dep_rel</code> property on TOKEN; label on DEP edge
<code>token.head</code> (parser output)	Syntactic head of each token	DEP edges from head token to dependent
<code>token.ent_type_</code> , <code>token.ent_iob_</code>	NER tag and IOB position	properties on TOKEN for downstream entity span construction
<code>sent.ents</code>	Named entity spans	ENTITY nodes; CONTAINS from sentence; BELONGS_TO from entity to constituent tokens
Content verbs (<code>token.pos_</code> in VERB / AUX filter)	Predicate-argument extraction	PREDICATE nodes; CONTAINS from sentence; BELONGS_TO to verb token
Predicate children with mapped dependency labels (<code>nsubj</code> , <code>dobj</code> , etc.)	Semantic role assignment	ARGUMENT nodes; SRL_ARG edges from token and from argument node to predicate
<code>sent.noun_chunks</code>	Noun-phrase chunking	NOUN_PHRASE nodes; CONTAINS from sentence; BELONGS_TO from chunk to its tokens
Clausal dependency labels (<code>advcl</code> , <code>relcl</code> , <code>ccomp</code> , <code>acl</code>)	Clause detection	CLAUSE nodes
Verb-phrase dependents (auxiliaries, negation, particles, adverbial modifiers)	Verb-phrase chunking	VERB_PHRASE nodes
Token order within a sentence	Linear flow	NEXT_TOKEN edges
Sentence order within a paragraph	Linear flow	NEXT_SENT edges
Paragraph order within the document	Linear flow	NEXT_PARA edges
Last token of sentence N to first token of sentence $N+1$	Cross-sentence continuity	extra NEXT_TOKEN edges

Table 3: spaCy annotations used by the TPG generic frontend, and what the frontend produces from each one.

Edge type	Source role	Target role
SEC_AFFECTS	CVE / vulnerability	SOFTWARE
SEC_HAS_VERSION	SOFTWARE	VERSION
SEC_LOCATED_IN	VULN_TYPE	CODE_ELEMENT
SEC_CLASSIFIED_AS	CVE	CWE
SEC_EXPLOITED_BY	CVE / vulnerability	ATTACK_VECTOR
SEC_CAUSES	ATTACK_VECTOR	IMPACT
SEC_MITIGATED_BY	CVE / vulnerability	REMEDIATION
SEC_USES_FUNCTION	CVE / vulnerability	CODE_ELEMENT
SEC_THREATENS	ATTACK_VECTOR	SOFTWARE
SEC_HAS_SEVERITY	CVE	SEVERITY

Table 4: The ten `SEC_*` edge types emitted by the security relations pass.

Hyperparameter	CLI flag	Default
GNN backbone	<code>-backbone</code>	<code>multiview</code> (we used this; alternatives: <code>gcn</code> , <code>gat</code> , <code>sage</code> , <code>edge_type</code> , <code>r</code>)
Hidden dimension	<code>-hidden</code>	128
Number of GNN layers	<code>-layers</code>	3 (the multiview backbone defaults to 4 GGNN unrolling steps internally)
Attention heads (GAT/RGAT)	<code>-heads</code>	4
Dropout	<code>-dropout</code>	0.3
Epochs	<code>-epochs</code>	100 (we set this explicitly)
Batch size	<code>-batch-size</code>	32
Learning rate	<code>-lr</code>	1×10^{-3}
Weight decay	<code>-weight-decay</code>	1×10^{-4}
Early-stopping patience	<code>-patience</code>	15 epochs without val PR-AUC improvement
Random seed	<code>-seed</code>	42
SecBERT embedding dim	<code>-embedding-dim</code>	768
Top-K CWEs encoded	<code>-top-k-cwes</code>	25

Table 5: Hyperparameters and their defaults.

Quantity	Value
Node feature dimension d_{in}	781 (13-dim node-type one-hot + 768-dim SecBERT)
Tabular feature dimension	55 (with EPSS removed)
Number of edge types	13 (without security edges) or 23 (with)
Hidden width h	128
Tabular MLP width h_t	64
Pooled graph embedding	$2h = 256$ (mean concatenated with max)
Fusion vector $z = [g; t]$	$2h + h_t = 320$

Table 6: Tensor dimensions in the leakage-free configuration.